# 08030f89-48e0-4052-9704-8e8327ff2d

## <prev_user>

### <request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'github/gh-aw' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/gh-aw/gh-aw (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-04-04T15:31:29.797Z</current_datetime>

Consider the following problem statement:

<problem_statement>

----
*This section includes details on the pull request with code changes you have already submitted to fix the problem above.*

<pr_title>
Fix repo-root-relative import path resolution in `ResolveIncludePath`
</pr_title>

<pr_description>
`imports: [.github/agents/planner.md]` was broken because `ResolveIncludePath` unconditionally joined import paths onto `baseDir` (`.github/workflows/`), producing `.github/workflows/.github/agents/planner.md` — a path that never exists.

## Changes

- **`pkg/parser/remote_fetch.go` / `remote_fetch_wasm.go`**: Before resolving, detect repo-root-relative paths and adjust the resolution base accordingly:
  - `.github/`-prefixed → resolve from repo root; security scope

stays `.github/`
  - `/`-prefixed → strip leading slash, resolve from repo root; security scope is repo root (still blocks traversal above it)
  - All other paths → unchanged behavior (resolved relative to `baseDir`)

- **`pkg/parser/frontmatter_utils_test.go`**: Added test cases covering `.github/agents/` imports, `/`-prefixed repo-root imports, backward-compat relative imports, and path traversal rejection for both new code paths. Also added `TestResolveIncludePath_DotGithubRepo` verifying all three resolution rules and traversal rejection work correctly when the repository itself is named `.github` (e.g. an org's `org/.github` repo), where the on-disk layout is `<parent>/.github/.github/workflows/`.

## Example

```yaml
# Previously failed (resolved to .github/workflows/.github/agents/planner.md)
imports: [.github/agents/planner.md]

# Now resolves correctly to <repo-root>/.github/agents/planner.md
# Also supported:
imports: [/agents/agent.md]  # resolves to <repo-root>/agents/agent.md
```

> [!WARNING]
> The security scope for `/`-prefixed paths is currently the entire repo root. A follow-up may restrict this to only `.github/` and `.agents/` subdirectories — feedback is pending on the intended security boundary.
</pr_description>

## Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>@pelikhan</author>
@copilot add tests to check that these rules work in the repo name ".github" as well

</comment_old>

<comment_old>
<author>@pelikhan</author>
@copilot only the .github, .agents folder are available when resolving imports. Other repo root path will be resolve
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot add tests to check that these rules work in the repo name ".github" as well

Added `TestResolveIncludePath_DotGithubRepo` in commit `2da3602`. It simulates a repo whose root is a directory ending in `.github` (e.g. `org/.github`), with the layout `<parent>/.github/.github/workflows/`, and verifies that:

- relative paths still resolve within the workflows dir
- `.github/agents/planner.md` resolves from the repo root to `<parent>/.github/.github/agents/planner.md`
- `/agents/agent.md` resolves from the repo root to `<parent>/.github/agents/agent.md`
- traversal attacks via both prefixes are rejected
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot only the .github, .agents folder are available when resolving imports. Other repo root path will be resolve

The comment appears truncated — could you clarify the intended behavior? Specifically:

1. Should `/`-prefixed paths (e.g. `/agents/agent.md`) be restricted to only `.github/` and `.agents/` subfolders at the repo root, with paths outside those two folders being rejected?
2. Or should the current behavior (entire repo root is accessible via `/`-prefix) remain, and this comment is just describing a different constraint?

If (1), I'll update the security scope logic in `ResolveIncludePath` to only allow `.github/` and `.agents/` for `/`-prefixed paths, and update the tests accordingly.
</comment_old>

```
<comment_new>
<comment_id>4187274775</comment_id>
<author>@pelikhan</author>
@copilot restrict workspace root to .github and .agents
</comment_new>
```

</pr_comments>

</comments>

---
The last **4** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.

## <prev_assistant>

Here's a summary of what was done to address comment `4187274775`:

**Change**: Restricted `/`-prefixed workspace-root import paths to only `.github/` and `.agents/` subdirectories.

**`remote_fetch.go` and `remote_fetch_wasm.go`**: The `else if` branch handling `/`-prefixed paths now:
1. Immediately rejects any path whose stripped form doesn't start with `.github/` or `.agents/` (e.g. `/agents/agent.md` → security error)
2. Sets `securityBase = filepath.Join(repoRoot, ".agents")` for `.agents/`-prefixed paths
3. Explicitly sets `securityBase = githubFolder` for `.github/`-prefixed paths (clarity fix from code review)

**Tests**: Updated both `TestResolveIncludePath` and `TestResolveIncludePath_DotGithubRepo` to:
- Replace the `<repoRoot>/agents/` fixture with `<repoRoot>/.agents/`
- Add `/.github/...` and `/.agents/...` success cases
- Add a rejection test for `/agents/agent.md` (non-allowlisted path)

## <current>

### <request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'github/gh-aw' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/gh-aw/gh-aw (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-04-04T16:27:47.724Z</current_datetime>

Consider the following problem statement:

<problem_statement>

---
*This section includes details on the pull request with code changes you have already submitted to fix the problem above.*

````
<pr_title>
Fix repo-root-relative import path resolution in
`ResolveIncludePath`
</pr_title>

<pr_description>
`imports: [.github/agents/planner.md]` was broken because
`ResolveIncludePath` unconditionally joined import paths onto
`baseDir` (`.github/workflows/`), producing `.github/
workflows/.github/agents/planner.md` — a path that never exists.

## Changes

- **`pkg/parser/remote_fetch.go` / `remote_fetch_wasm.go`**:
Before resolving, detect repo-root-relative paths and adjust the
resolution base accordingly:
  - `.github/`-prefixed → resolve from repo root; security scope
stays `.github/`
  - `/`-prefixed → strip leading slash, resolve from repo root;
only `.github/` and `.agents/` subdirectories are accessible
(all other paths are rejected with a security error)
  - All other paths → unchanged behavior (resolved relative to
`baseDir`)

- **`pkg/parser/frontmatter_utils_test.go`**: Added test cases
covering `.github/agents/` imports, `/.github/`- and `/.agents/
`-prefixed repo-root imports, backward-compat relative imports,
path traversal rejection, and rejection of `/`-prefixed paths
outside `.github/` or `.agents/`. Also added
`TestResolveIncludePath_DotGithubRepo` verifying all resolution
rules and traversal rejection work correctly when the repository
itself is named `.github` (e.g. an org's `org/.github` repo),
where the on-disk layout is `<parent>/.github/.github/workflows/
`.

## Example

```yaml
# Previously failed (resolved to .github/workflows/.github/
agents/planner.md)
imports: [.github/agents/planner.md]

# Now resolves correctly to <repo-root>/.github/agents/
planner.md
# Also supported (restricted to .github/ and .agents/ only):
imports: [/.github/agents/planner.md]  # resolves to <repo-
````
````

```
root>/.github/agents/planner.md
imports: [/.agents/agent.md]           # resolves to <repo-
root>/.agents/agent.md

# Rejected — only .github/ and .agents/ are accessible from repo
root:
imports: [/agents/agent.md]  # security error
```
</pr_description>

## Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>@pelikhan</author>
@copilot add tests to check that these rules work in the repo
name ".github" as well
</comment_old>

<comment_old>
<author>@pelikhan</author>
@copilot only the .github, .agents folder are available when
resolving imports. Other repo root path will be resolve
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot add tests to check that these rules work in the
repo name ".github" as well

Added `TestResolveIncludePath_DotGithubRepo` in commit
`2da3602`. It simulates a repo whose root is a directory ending
in `.github` (e.g. `org/.github`), with the layout
`<parent>/.github/.github/workflows/`, and verifies that:

- relative paths still resolve within the workflows dir
- `.github/agents/planner.md` resolves from the repo root to
`<parent>/.github/.github/agents/planner.md`
- `/agents/agent.md` resolves from the repo root to
`<parent>/.github/agents/agent.md`
- traversal attacks via both prefixes are rejected
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot only the .github, .agents folder are available when resolving imports. Other repo root path will be resolve

The comment appears truncated — could you clarify the intended behavior? Specifically:

1. Should `/`-prefixed paths (e.g. `/agents/agent.md`) be restricted to only `.github/` and `.agents/` subfolders at the repo root, with paths outside those two folders being rejected?
2. Or should the current behavior (entire repo root is accessible via `/`-prefix) remain, and this comment is just describing a different constraint?

If (1), I'll update the security scope logic in `ResolveIncludePath` to only allow `.github/` and `.agents/` for `/`-prefixed paths, and update the tests accordingly.
</comment_old>

<comment_old>
<author>@pelikhan</author>
@copilot restrict workspace root to .github and .agents
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot restrict workspace root to .github and .agents

Done in commit `cd24226`. `/`-prefixed workspace-root imports are now restricted to `.github/` and `.agents/` only — any other path (e.g. `/agents/agent.md`) is immediately rejected with a security error. Tests updated accordingly.

> [!WARNING]
>
></comment_old>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
## Pull request overview

Fixes local include/import resolution so repo-root-relative paths (notably `.github/...` and `/.github/...`) resolve from the repository root instead of being incorrectly joined onto the

```
workflow’s `baseDir` (e.g. `.github/workflows/...`). Also
introduces controlled access to repo-root `/.agents/...` paths
and adds tests for the new resolution and security rules.

**Changes:**
- Update `ResolveIncludePath` to treat `.github/...` and `/
<repo-root>...` paths as repo-root-relative with an explicit
security scope.
- Mirror the same path-resolution behavior in the Wasm build.
- Add unit tests covering `.github/agents` imports, `/.github` /
`/.agents` imports, traversal rejection, and the special case
where the repository itself is named `.github`.



## Copilot's findings

&gt; [!TIP]
&gt; <a href="/github/gh-aw/new/main?filename=.github/
instructions/*.instructions.md" class="Link--inTextBlock"
target="_blank" rel="noopener noreferrer">Add Copilot custom
instructions</a> for smarter, more guided reviews. <a
href="https://docs.github.com/en/copilot/customizing-copilot/
adding-repository-custom-instructions-for-github-copilot"
class="Link--inTextBlock" target="_blank" rel="noopener
noreferrer">Learn how to get started</a>.


<details>
<summary>Comments suppressed due to low confidence (1)</summary>

**pkg/parser/remote_fetch.go:174**
* Repo...
<__filter_complete__></__filter_complete__></details></
comment_old>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>pkg/parser/remote_fetch.go:139-147</file>
<original_commit>cd24226</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
The `.github`-folder search loop can become non-terminating on
```

Windows when `baseDir` is not under a `.github` directory (e.g., `C:\...`), because `filepath.Dir("C:\\") == "C:\\"` and the loop only stops on "." or "/". Add a cross-platform termination condition (e.g., break when the parent dir equals the current dir, or handle volume roots) so local includes from non-.github directories don’t hang.

This issue also appears on line 150 of the same file.

See below for a potential fix:
```
                parent := filepath.Dir(githubFolder)
                if parent == githubFolder || parent == "." ||
parent == "/" {
                        // If we can't find .github folder, use
baseDir
                        githubFolder = baseDir
                        break
                }
                githubFolder = parent
```
</comment_old>

</comment_thread>

<comment_thread>
<file>pkg/parser/remote_fetch.go:187-188</file>
<original_commit>cd24226</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
The security escape error message still says "must be within .github folder" even when `securityBase` is `.agents` (for `/.agents/...` imports). Update the error text to reflect the actual allowed base (e.g., mention `.agents` when applicable, or use a generic "allowed folder" message) so users aren’t misled during debugging.
```suggestion
                allowedFolder :=
filepath.Base(normalizedSecurityBase)
                remoteLog.Printf("Security: Path escapes allowed
folder: %s (resolves to: %s)", filePath, relativePath)
                return "", fmt.Errorf("security: path %s must be
within %s folder (resolves to: %s)", filePath, allowedFolder,
relativePath)
```
</comment_old>

</comment_thread>

<comment_thread>
<file>pkg/parser/remote_fetch_wasm.go:109</file>
<original_commit>cd24226</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
In the Wasm implementation, the security escape error message always says "must be within .github folder" even though `securityBase` can be `.agents` for `/.agents/...` imports. Adjust the error text to reflect the actual security scope (or use a generic "allowed folder" message) to avoid misleading errors.
```suggestion
                return "", fmt.Errorf("security: path %s must be within the allowed folder (resolves to: %s)", filePath, relativePath)
```
</comment_old>

</comment_thread>

<comment_thread>
<file>pkg/parser/remote_fetch_wasm.go:80-99</file>
<original_commit>cd24226</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
Repo-root-relative detection here only recognizes POSIX-style prefixes (`.github/`, `.agents/`, leading `/`). If callers pass Windows-style paths (e.g., `.github\\agents\\planner.md`), the repo-root-relative logic won’t trigger. Consider normalizing `filePath` (e.g., `filepath.ToSlash`) for these prefix checks or handling `\\` variants explicitly, consistent with other path-normalization code in the repo.
</comment_old>

</comment_thread>

</pr_reviews>

<pr_comments>

<comment_new>
<comment_id>4187352727</comment_id>
<author>@pelikhan</author>

```
@copilot View comments
</comment_new>

</pr_comments>

</comments>

----
The last **6** git commits in this branch are the changes you
have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the
<comments> before making any changes. You will only address the
<comment_new> comments. Reply to new comments as you address them.
Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a
suggestion.
    - Is it addressed to you or someone else, e.g. "Hey
@username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can
stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and
ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

A **CI/Build Failure** has been mentioned in the problem statement,
make sure to follow your instructions for addressing CI/Build
Failures.
```